Project Report

Submitted by
Shaheer Fardan
Udacity AWS Machine Learning Engineer Nanodegree Program

Capstone Project: Inventory Monitoring at Distribution Centers

Definition

Project Overview: In distribution Centers like in that of Amazon/Flipkart, robots are often used to move objects between different warehouse locations as part of their day-to-day operations. Different objects depending on their size are clubbed together in a single bin for efficient movement of objects. So, a single bin can carry one, two three, four or five objects (or even more). This project aims to build a Computer Vision Model that can be used to count number of objects in each bin. A system like this can be used to track inventory and make sure that delivery consignments have the correct number of items.

Problem Statement: You are provided with different image files containing different pictures of bins. These bins can carry from 1-5 objects. Problem involves counting number of objects in each of the bin images irrespective of type, shape and packaging of the objects. Such problems will be encountered by any distribution center using such techniques to move objects from one place to another. So, problem is replicable, quantifiable and measurable.

Solution Statement: We need to have an automated system that can count number of objects present in bin as shown in an image file. This is basically a Computer Vision problem and can be solved in two ways: through object localization and detection and through classification. Our choice of solution is to use classification approach as such solutions are easy to train and implement. We will be using pre-trained CNN models for this purpose. We will freeze the CNN model, attach a new classification head on top of it and train only the classification head part of it using the provided dataset. Once the model is fine-tuned, it can be evaluated and then deployed to end-point where it can be used for inference. System will input an image file to the end-point. Model deployed at endpoint will perform classification and send number of objects present in the image back to calling system. Such a solution can be used in different distribution centers using similar technique to move around objects. We can also assess the error rate of model post deployment.

Metrics: We will be using accuracy to assess the model performance.

Accuracy = (number of images correctly classified)/(total number of images in dataset)

Due to resource constraints, we are using a subset of dataset and hence the metrics like recall/precision might not be appropriate here and may not present true picture with respect to full dataset so, accuracy is the right metric to present correct picture with respect to chosen dataset subset and duration for which model is trained.

Analysis

Data Exploration: We are going to use Amazon Bin Image Dataset which has 500,000 bin images containing one or more objects. Due to resource constraints, we will be using subset of dataset provided with project.

The subset contains following classes of image files followed by number of images belonging to each class. Total number of images in sample dataset is 10441.

Class 1: 1228

Class 2: 2299

Class 3: 2666

Class 4: 2373

Class 5: 1875

We will divide the dataset subset into training, validation and test datasets in following proportions stratified by image classes:

Train – 90% - 9396 images

Validation – 5% - 522 Images

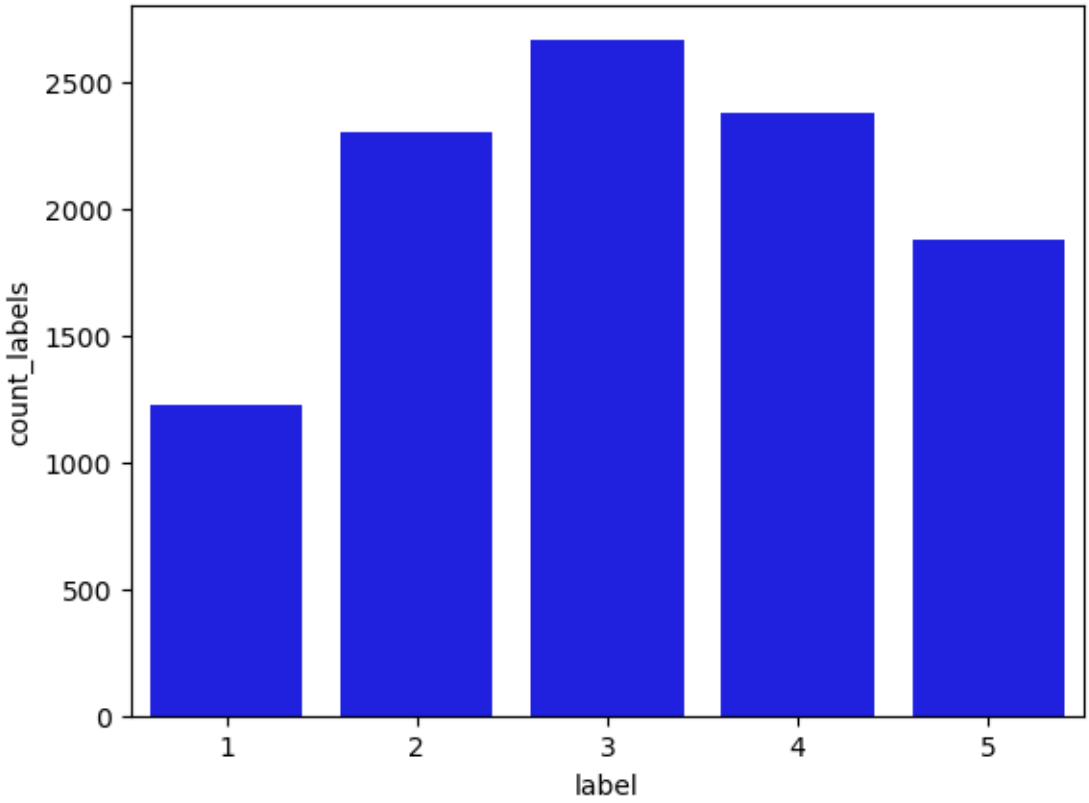
Test – 5% - 523 images

Details about full dataset can be found at https://registry.opendata.aws/amazon-bin-imagery/

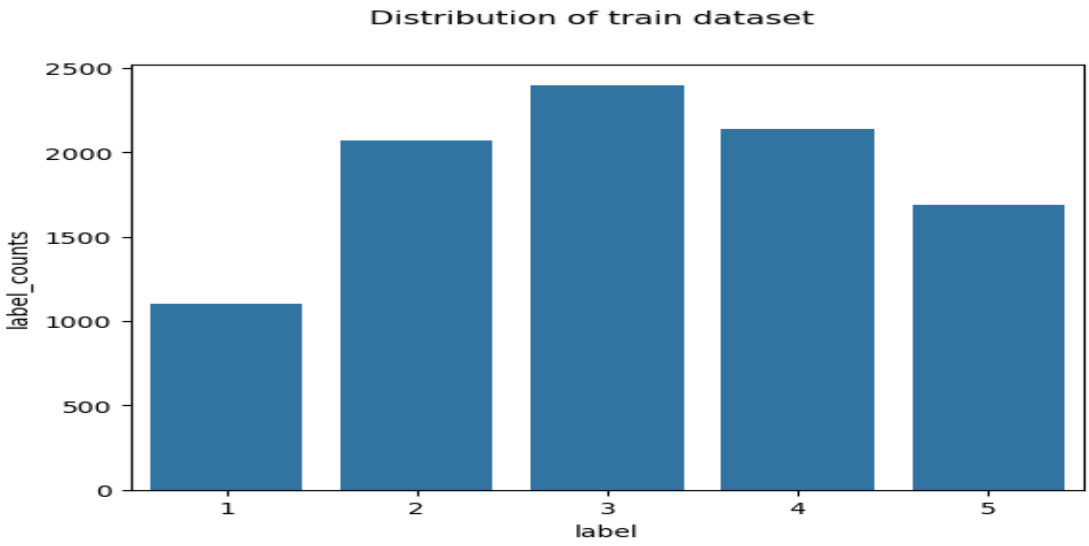
This project involves counting of objects present in a bin using image classification algorithm.

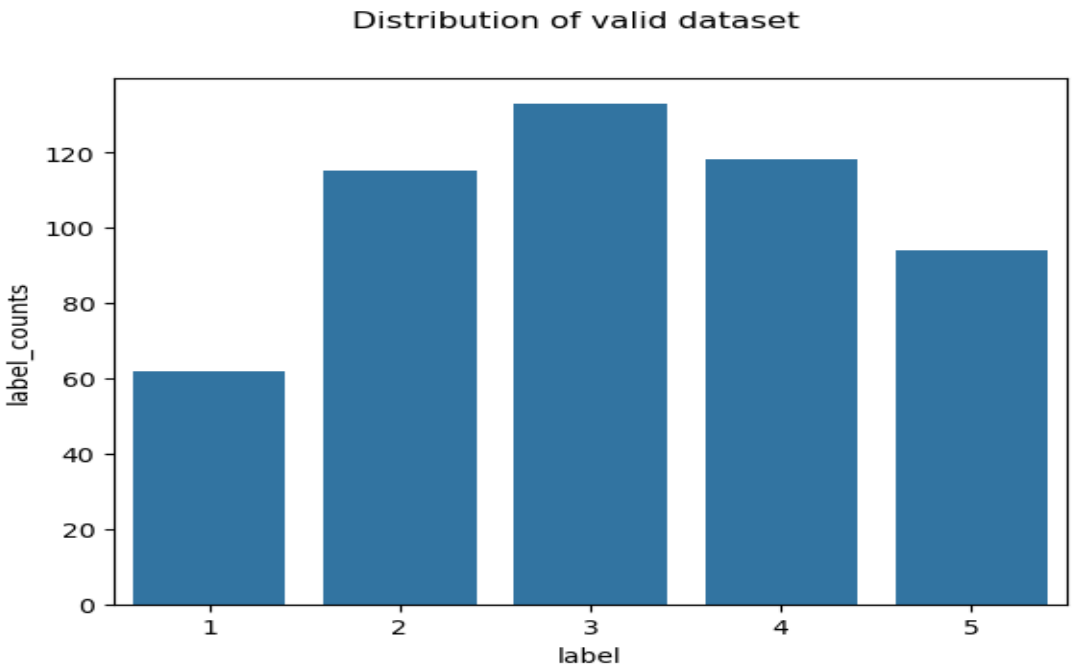
Exploratory Visualization

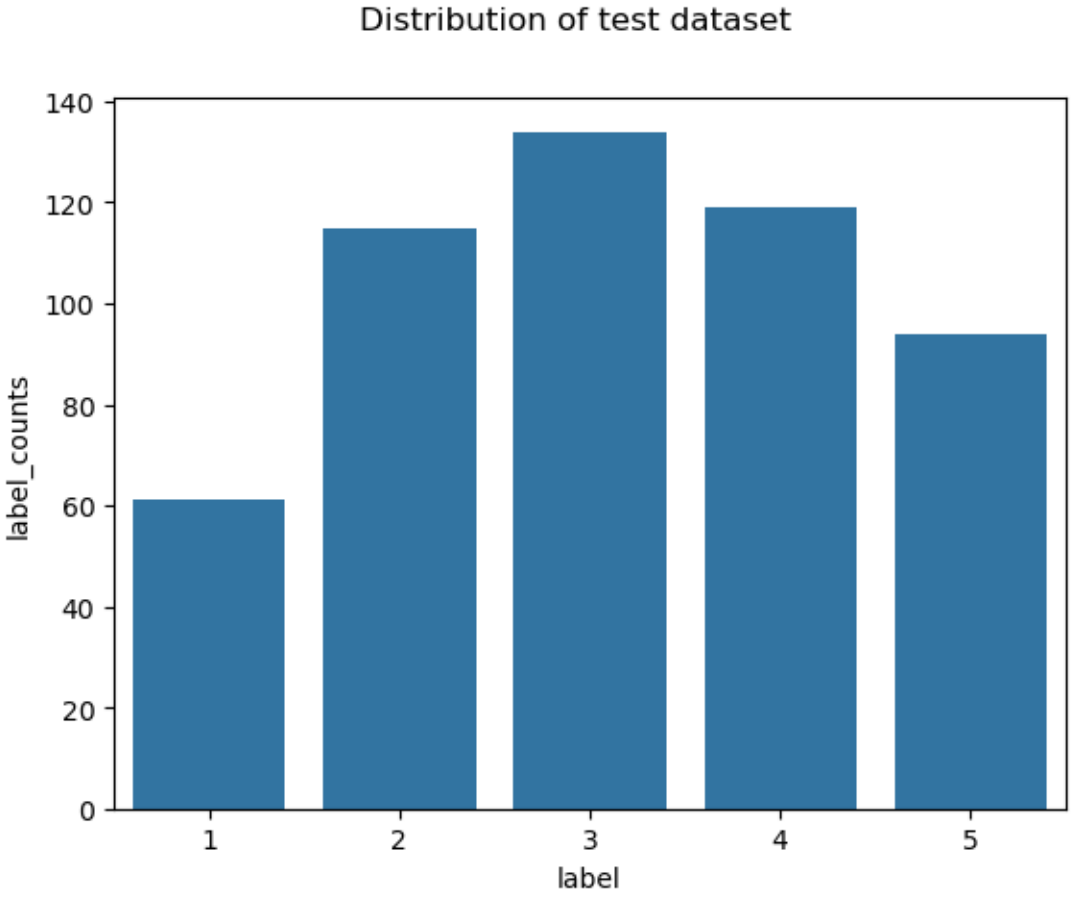
Dataset distribution prior to split:



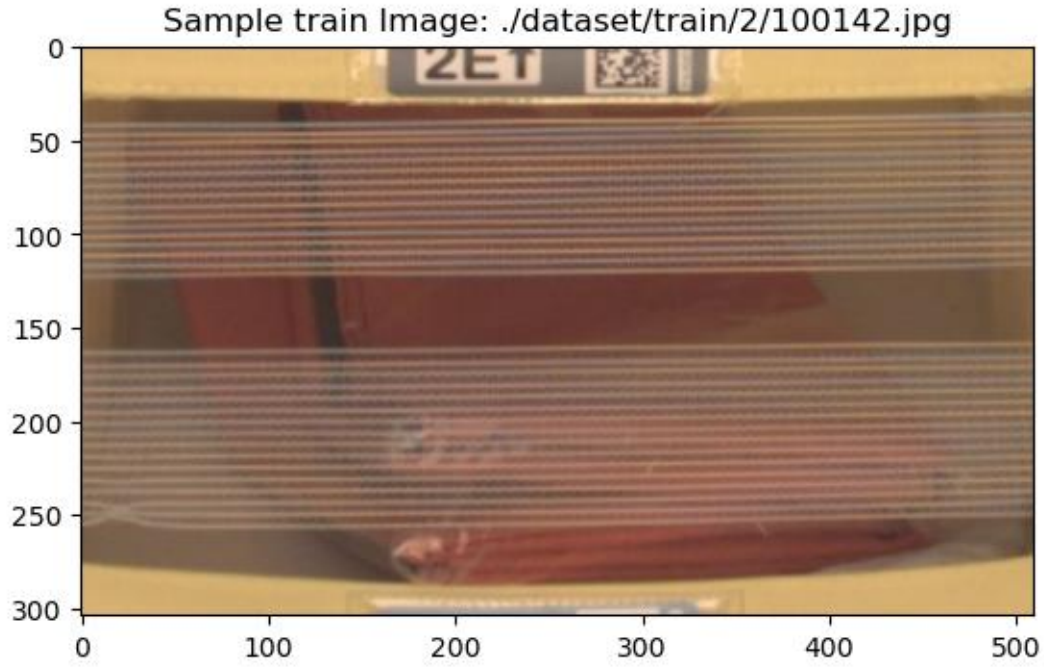
After distributing data in training/validation/test datasets, class image distributions look as under for different datasets:

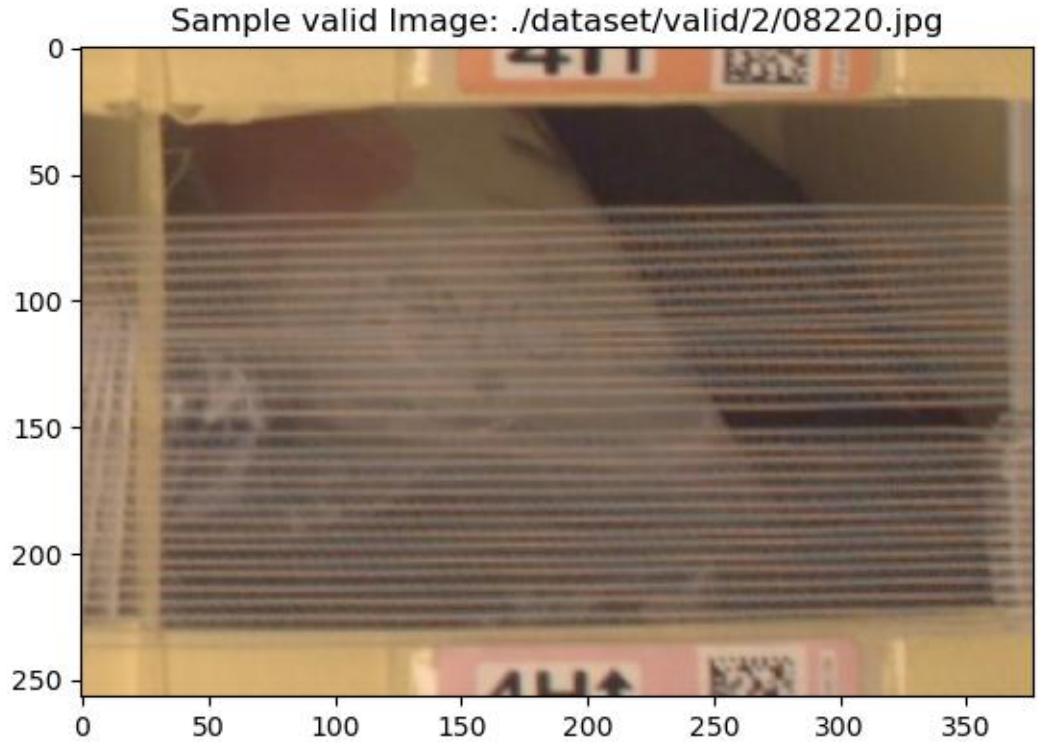


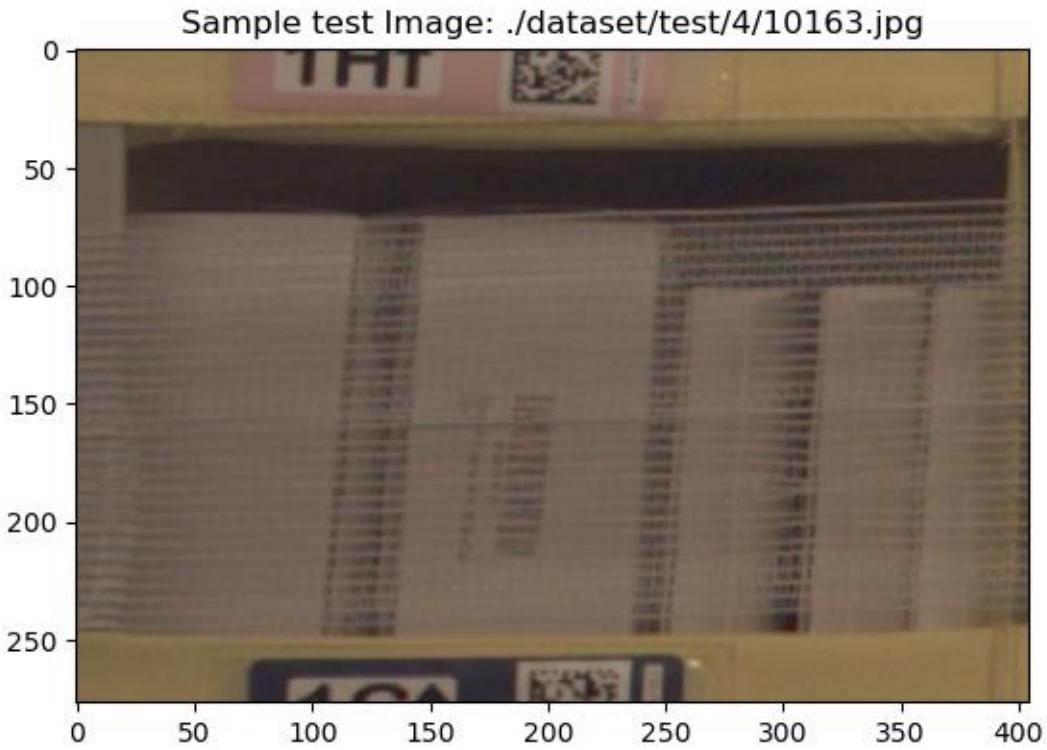

Distribution of valid dataset
label_counts
0
20
40
60
80
100
120
1
2
3
4
5
label


Distribution of test dataset
label_counts
0
20
40
60
80
100
120
140
1
2
3
4
5
label

Following are few sample images from training/validation/test dataset. Label numbers should be interpreted as n+1(eg, if 2 then actual is 3). This is because the label numbers were renumbered for zero indexing.







Algorithms and Techniques:

We have utilized CNN based architecture to solve the problem at hand. Specifically, we have used two SOTA models resnet50 and efficientnet_b0. When it comes to speed and accuracy, resnet50 combines best of both the worlds. Efficientnet_b0 is little behind with respect to accuracy however it is lightweight and highly performant model, more suitable to be deployed on small devices. Resnet50 is little bulky, however, considering the size of modern-day vision models based on transformers architecture, this doesn't seem to be a negative point for resnet50. Accordingly, efficientnet_b0 is fastest to train and infer. Resnet50 comes close.

So, with respect to speed, accuracy and inference times, these two models form good choice to start with. We train both the models using a predefined set of hyperparameters. Once the training is complete, we compare the validation/test accuracy on both the models. Model producing higher validation/test accuracy is chosen and we perform hyperparameter tuning on it. If validation accuracies are close then we chose the model with high test accuracy. Post hyperparameter tuning, we take the hyperparameters corresponding to best performing model checkpoint and train it further using sagemaker debugger and profiler options. We also use the same set of hyperparameters to perform multi-instance training.

Benchmark: We have trained model using subset of full dataset and chosen subset is unique to project, its little difficult to find a public benchmark to compare against. It will not be a fair comparison since we are not fully training the model due to resource constraints.
However, taking into account number of classes (five), a random guess says image can be classified into any of the classes with 20% probability (assuming equal distribution). So, a model producing more than 20% accuracy will be better than a random model. However, to set a bar little high, we decided any model producing 30% or more accuracy is a good model given the project setup and constraints. So, we set validation accuracy of 30% as a benchmark to compare the models.

Following is an example of validation/test accuracy reported during initial training run.

Initial run with set hyperparameters:

Resnet50:
Validation Accuracy: 0.3180
Test Accuracy: 0.3575

Efficientnet_b0:
Validation Accuracy: 0.3199
Test accuracy: 0.3212

Methodology

Data Preprocessing: Data Preprocessing is divided into multiple phases:

1) We take subsample of dataset and split it into train/validation/test datasets and then populate the original datafile location of images in a feature store. We do so by using a sagemaker processing job. This is to ensure reusability of dataset between different groups in an organization.
2) From feature group, we extract datafile locations of images belonging to training/validation and test datasets.
3) Next, we download the data from locations extracted in step 2 into local compute environment.
4) There are 5 image classes in dataset numbered from 1 to 5. Now issue here is softmax indexing is zero based so we need to renumber image classes to n-1(from 0 to 4). Accordingly, image class folders are renumbered under each data split.
5) Next, we upload the data from local compute environment to our S3 location.
6) We create local pytorch dataset and dataloader and iterate through it to ensure there is no problem loading any of the image files during training or validation/testing process
7) During actual training of model, image files need to be preprocessed/transformed to match model requirements and to help model generalize better on unseen data. Following image preprocessing steps were used prior to feeding the model with data:
    a) Image is resized to 256, 256

b) Perform random affine transformation which is a combination of scale, translational and rotational variances. This is to introduce more image variety during training without increasing the dataset size. For example, a cat image rescaled to 0.9 of original size, placed at the corner of image and rotated 45% is still an image of cat.
c) Perform random horizontal flip with 50% probability
d) Next, we randomly crop the image to 224, 224 size
e) Next, Image is converted into pytorch tensor followed by pixel standardization/normalization

For validation and test data, we use only transformations specified in 7.a, 7.d and 7.e.

Please see below image for more details on the transformation applied to training/validation/test data. Please note that the same validation transforms are also performed on test data as well.

```
train_transforms = T.Compose([
    T.Resize(256),
    T.RandomAffine(scale=(0.9, 1.1), translate=(0.1, 0.1), degrees=10),
    T.RandomHorizontalFlip(0.5),
    T.RandomResizedCrop((224, 224)),
    T.ToTensor(),
    T.Normalize(mean=mean, std=std)
])

valid_transforms = T.Compose([
    T.Resize(256),
    T.CenterCrop((224, 224)),
    T.ToTensor(),
    T.Normalize(mean=mean, std=std)
])
```

Implementation and Refinement: Post data preprocessing (Steps 1-6), implementation is carried out as follows:

1) We create docker image integrating our training script(train.py) and inference scripts for different models(resnet50 and efficientnet_b0) along with it. We push the docker image to ECR and use its ECR Image path for training jobs. Since we already have training container, we can carry out training using a normal Estimator object. Please note that train.py has been written in such a way that it can be used with static hyperparameters, hyperparameter ranges and with/without sagemaker debugger/profiler. So, the same train.py is used in all training jobs.
2) We train resnet50 and efficientnet_b0 models using predefined hyperparameters.

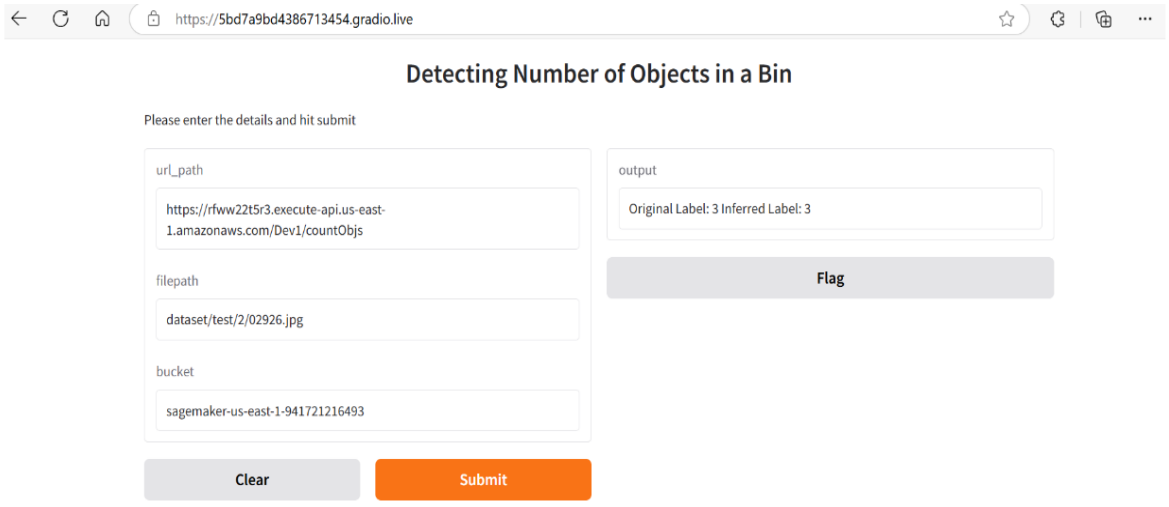
3) We compare their validation/test accuracies. Model with higher validation/test accuracy is used in further trainings and the other one is dropped. Here, resnet50 wins the competition
4) Next we define hyperparameter ranges and perform hyperparameter tuning on resnet50 model.
5) We determine the best hyper parameters from the training subjob showing highest validation accuracy.
6) We take the best hyper parameters from step 5 and retrain resnet50 with sagemaker debugger and profiler options enabled
7) Next we use the same set of hyperparameters and perform muti-instance training using resnet50 model
8) Out of all the jobs (in steps 4 to 7), multi-instance training from step 7 provides the best performing model.
9) We extract the model path in step 7/8 and use it for further deployment
10) Next we perform single model endpoint deployment, multi-model endpoint deployment, multi-model endpoint deployment using production variants
11) For multi-model endpoint deployment along with resnet50 model checkpoint, we use efficientnet_b0 model checkpoint. Multi-model checkpoint is used to save cost by serving multiple models out of one container for inference. Inference request will contain the model name it wants to infer against, container will load corresponding model and perform inference.
12) For multi-model endpoint deployment using production variants, we can use same model checkpoint or different checkpoints or different model checkpoint. This setup can be used for canary deployment or A/B testing scenario. Variants can be added and taken out in real time without causing any downtime.
13) For production variant type endpoint, we initially configure both variants with equal weightage which means both variants will receive the inference requests in equal proportions. Next, we turn the weightage of variant1 to 0 and that of variant2 to 100% allowing all the traffic to go to only variant2 in real time. Next, we configure autoscaling on variant2 and watch the number of instances going up to 2 on account of increased traffic.
14) For all types of endpoints configured above we make sure to test the endpoint by sending inference traffic to corresponding endpoint.
15) Next, we configured a sagemaker pipeline that will carry out training, evaluation, model registration, model creation in an automated fashion. Next, we approve the model and deploy it and test out the endpoint by sending inference traffic to it.
16) For final testing, we configure a function to bring up endpoint (using the multi-instance resnet50 checkpoint). Next, we create a lambda function that will be used to invoke endpoint. Next, we integrate AWS API Gateway with lambda to perform inference on trained model. We create local function which is used to connect to AWS API Gateway URL and use it to perform inference. Next, we generate test case and use it along with

local function to perform inference. Further, we create gradio interface that is used in conjunction with local function to perform inference.

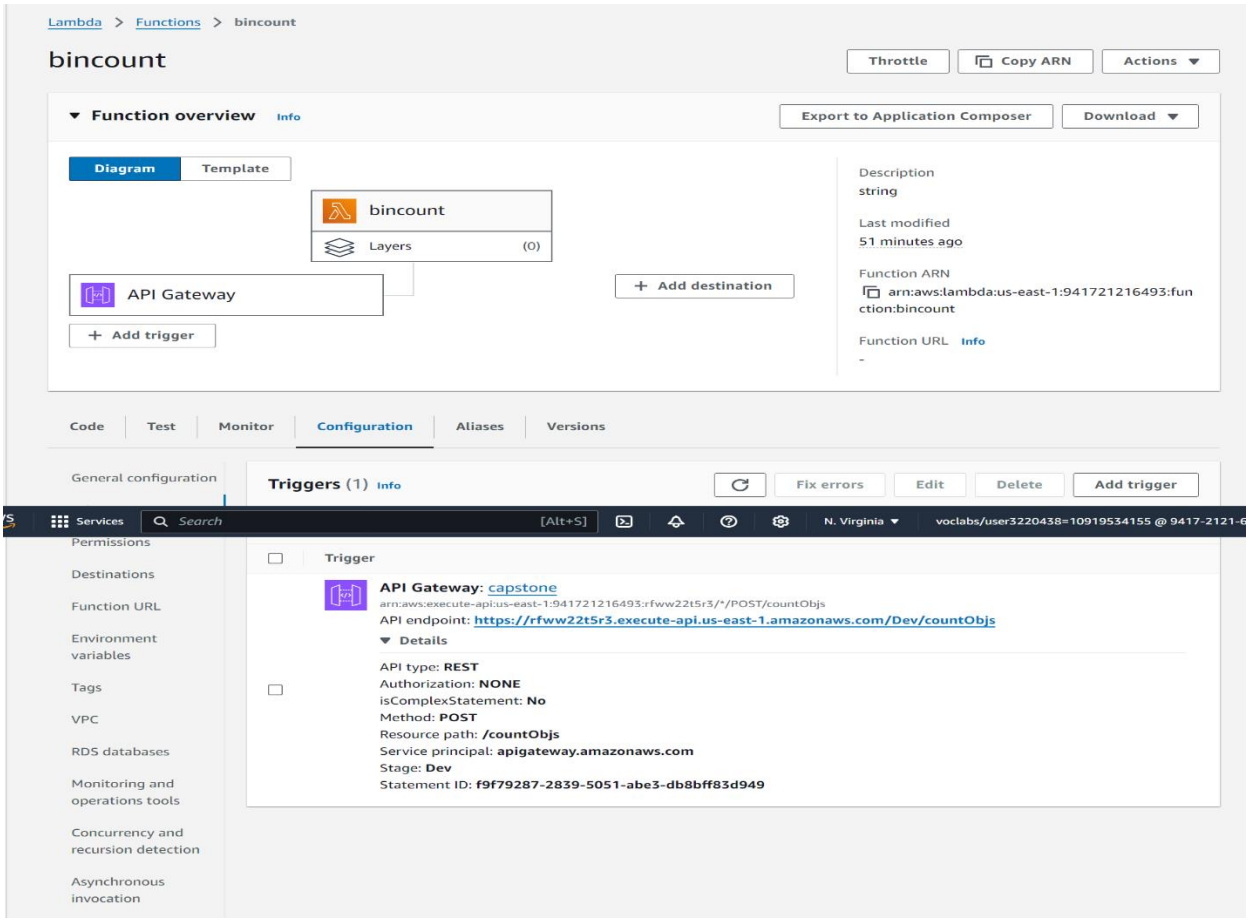
Inference process:

1. **Start** → Bring up the endpoint
2. **Generate Test Case**
3. **Bring Up Gradio Interface**
4. **Fill in Information**
    - Inference URL
    - Bucket
    - Image File Path
5. **Gradio Invokes Local Function**
    - Connects to Inference URL
6. **Inference URL Invokes Lambda Function**
7. **Lambda Function Invokes Endpoint**
8. **Endpoint Model Performs Inference**
    - Sends Softmax Array to Lambda Function
9. **Lambda Function Sends Inferred Information Back**
    - Through API Gateway to Local Function
10. **Local Function Performs Argmax on Returned Array**
11. **Function Sends Inferred Label & Original Label to Gradio Display**
12. **End**

Screenshot from outside AWS network using public URL



Screenshot representing integration of lamba function and AWS API Gateway



Multi-Model endpoint screenshot:

```
In [86]: !aws sagemaker list-endpoints
```


```
{
    "Endpoints": [
        {
            "EndpointName": "multi-model-endpt",
            "EndpointArn": "arn:aws:sagemaker:us-east-1:941721216493:endpoint/multi-model-endpt",
            "CreationTime": 1729940849.991,
            "LastModifiedTime": 1729941135.452,
            "EndpointStatus": "InService"
        }
    ]
}
```

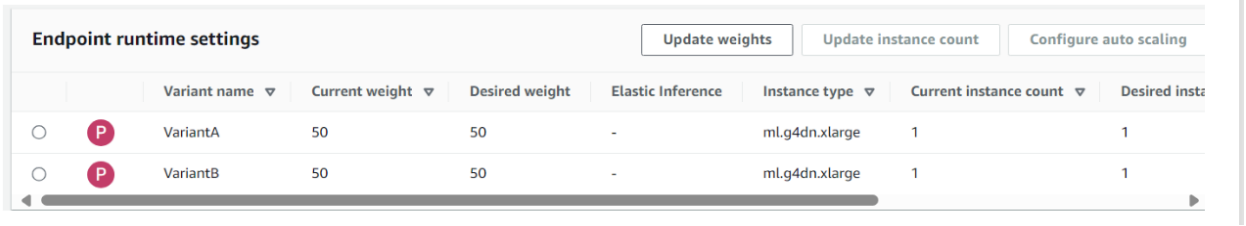

```
8]: a, b = list(multi_model.list_models())
    a, b
```

```
8]: ('capstone/model_data/model_a.tar.gz', 'capstone/model_data/model_b.tar.gz')
```

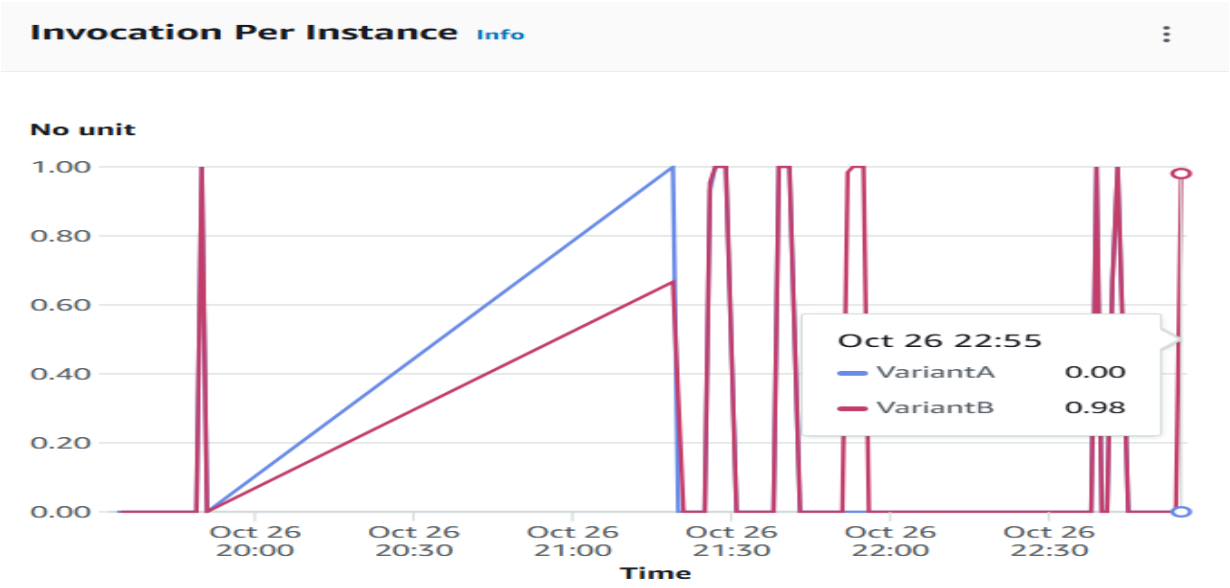
Multi-Model endpoint containing different production variants:

```
: !aws sagemaker list-endpoints
```

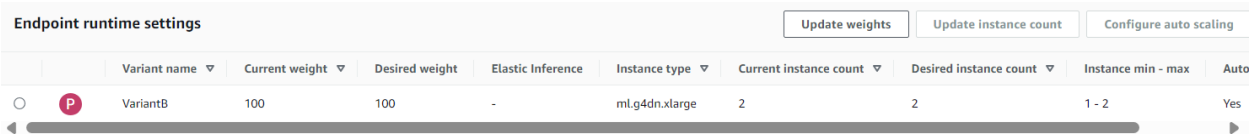

```
{
    "Endpoints": [
        {
            "EndpointName": "ab-endpoint",
            "EndpointArn": "arn:aws:sagemaker:us-east-1:941721216493:endpoint/ab-endpoint",
            "CreationTime": 1729962137.945,
            "LastModifiedTime": 1729962449.951,
            "EndpointStatus": "InService"
        }
    ]
}
```

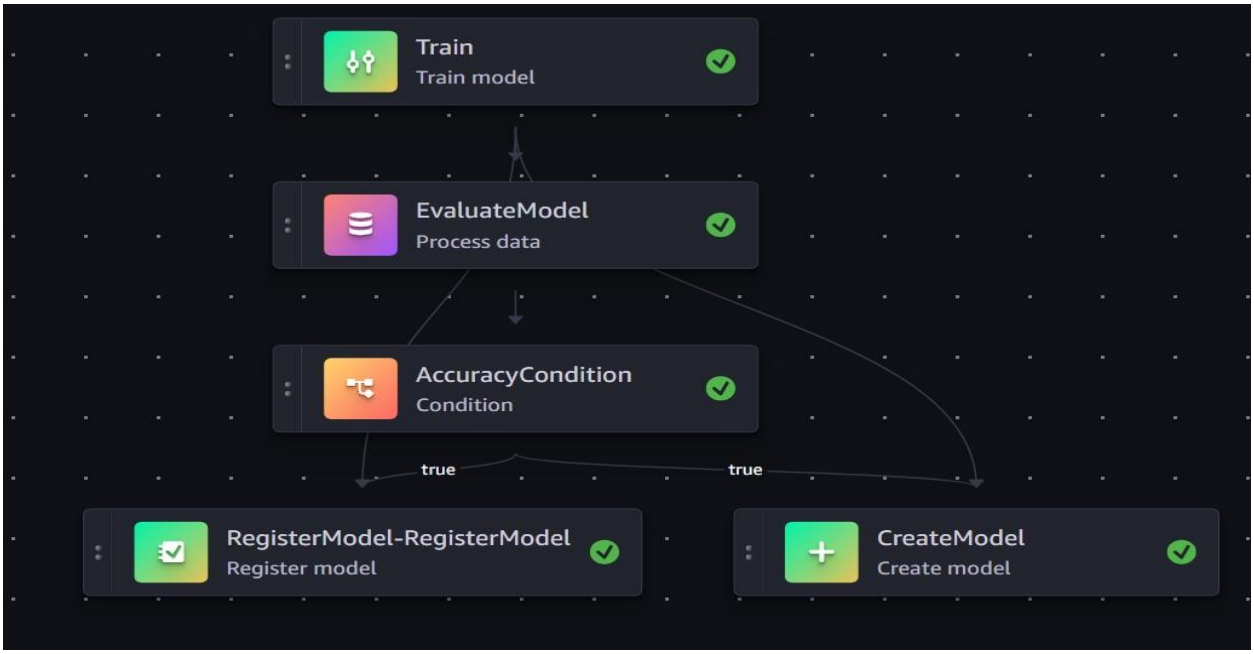
Screenshot when all the traffic was switched to one variant only



Endpoint Rescaling



Pipeline Execution Flowchart:



Results

Model Evaluation, Validation and Justification: As mentioned in the capstone proposal document and stated earlier in this report, accuracy was chosen as sole criterion for determination of model performance. Reasons also have been explained in a previous section and in proposal document.
Please see below validation and test accuracy reported during different phases of the project.

Run 1: Initial run with set hyperparameters:

Resnet50:
Validation Accuracy: 0.3180
Test Accuracy: 0.3575

Efficientnet_b0:
Validation Accuracy: 0.3199
Test accuracy: 0.3212

Here, we chose resnet50 because validation accuracy of both the models are very close, however, test accuracy of resnet50 exceeds that of efficientnet_b0 significantly.

Run 2: Hyperparameter tuning results:

Resnet50:
Validation Accuracy: 0.32759

Test Accuracy: 0.34417

Run 3: Training Run with debugger/profiler:

Resnet50:
Validation Accuracy: 0.31992
Test Accuracy: 0.35564

Run 4: Multi Instance Training Run:

Resnet50:
Validation accuracy: 0.31034
Test accuracy: 0.38432

Since the test accuracy of multi-instance model is best, we chose this model checkpoint for final endpoint deployment in different configurations.

As stated earlier in the document, the measuring benchmark that we are using is any model performing better than 30% validation/test accuracy is a good model given project constraints. As we can see during runs 2,3 and 4 validation accuracy of model is approximately in a very close range, however, test accuracy of model trained during run 4 beats others by a significant margin. So, we can say that resnet50 model trained during run 4 is the best performing model and can be used to adequately solve the problem within given constraints.